The `processRequestMessages` function processes incoming chat messages and adds an image to the last message if an image URL is provided in the request data. We can now use this function to process the request messages before providing them to the `streamText` function. The following listing shows the small snippet of code that provides the updated code.

Listing 3.15 Updating route.js to accept image prompts

```
export async function POST(req) {
    const messages = await processRequestMessages(req);
                const result = await streamText({
                model: model('models/gemini-2.0-flash'),
                maxTokens: 512,
                messages: [
                  {
                    role: 'system',
                    content: "I'm happy to assist you
in any way I can. How can I be of service today?",
                  },
                  ... messages,
                ],
  });
```

Calls the processRequestMessages to update the prompt messages to include image prompts

Passes on the messages to the streamText request

That completes the most straightforward part of the implementation. The next section is more challenging, as it involves creating a file uploader and file previewer in JavaScript. To keep things simple, we'll focus on the key concepts and provide essential snippets. The complete code can be reviewed at your convenience in the provided repository.

Updating the frontend to support uploading and previewing images, which will be sent along with the text prompts, involves three main steps:

1. *Creating the file uploader*—We need to create an input field that allows users to select and upload images. This will be accompanied by necessary validations to ensure the uploaded file meets the required criteria (e.g., file type and size).
2. *Previewing the image (optional)*—Once an image is uploaded, it should be displayed to the user for confirmation. This enhances the user experience by allowing them to see the image that will be sent along with the text prompt. However, this step is optional, as it does not hinder the next operation.
3. *Submitting the image with the prompt*—Finally, we need to modify the form submission logic to include the image data along with the text prompt. This will ensure that both text and images are sent to the backend for processing.

   For the code that implements the `FileUploader`, please review the implementation in the code sources accompanying this book. However, for our discussion, I focus solely on the logic for submitting the form with the image data along with the text prompt.

Let's modify the form submission logic in the `<Chat/>` component to include the image data along with the text prompt. You need to update the form `onSubmit` handler to use this new handler from the following listing.